14.1 Graphs

A ***graph*** is a way of representing relationships that exist between pairs of objects. That is, a graph is a set of objects, called vertices, together with a collection of pairwise connections between them, called edges. Graphs have applications in modeling many domains, including mapping, transportation, computer networks, and electrical engineering. By the way, this notion of a "graph" should not be confused with bar charts and function plots, as these kinds of "graphs" are unrelated to the topic of this chapter.

Viewed abstractly, a ***graph*** G is simply a set V of ***vertices*** and a collection E of pairs of vertices from V, called ***edges***. Thus, a graph is a way of representing connections or relationships between pairs of objects from some set V. Incidentally, some books use different terminology for graphs and refer to what we call vertices as ***nodes*** and what we call edges as ***arcs***. We use the terms "vertices" and "edges."

Edges in a graph are either ***directed*** or ***undirected***. An edge (u,v) is said to be ***directed*** from u to v if the pair (u,v) is ordered, with u preceding v. An edge (u,v) is said to be ***undirected*** if the pair (u,v) is not ordered. Undirected edges are sometimes denoted with set notation, as $\{u,v\}$, but for simplicity we use the pair notation (u,v), noting that in the undirected case (u,v) is the same as (v,u). Graphs are typically visualized by drawing the vertices as ovals or rectangles and the edges as segments or curves connecting pairs of ovals and rectangles. The following are some examples of directed and undirected graphs.

Example 14.1: *We can visualize collaborations among the researchers of a certain discipline by constructing a graph whose vertices are associated with the researchers themselves, and whose edges connect pairs of vertices associated with researchers who have coauthored a paper or book. (See Figure 14.1.) Such edges are undirected because coauthorship is a* **symmetric** *relation; that is, if A has coauthored something with B, then B necessarily has coauthored something with A.*

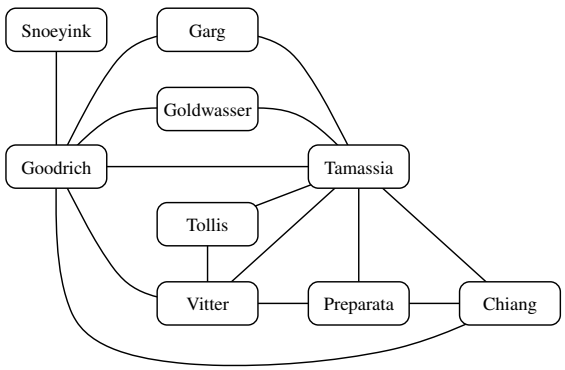


Figure 14.1: Graph of coauthorship among some authors.

Example 14.2: *We can associate with an object-oriented program a graph whose vertices represent the classes defined in the program, and whose edges indicate inheritance between classes. There is an edge from a vertex v to a vertex u if the class for v inherits from the class for u. Such edges are directed because the inheritance relation only goes in one direction (that is, it is* **asymmetric***).*

If all the edges in a graph are undirected, then we say the graph is an ***undirected graph***. Likewise, a ***directed graph***, also called a ***digraph***, is a graph whose edges are all directed. A graph that has both directed and undirected edges is often called a ***mixed graph***. Note that an undirected or mixed graph can be converted into a directed graph by replacing every undirected edge (u,v) by the pair of directed edges (u,v) and (v,u). It is often useful, however, to keep undirected and mixed graphs represented as they are, for such graphs have several applications, as in the following example.

Example 14.3: *A city map can be modeled as a graph whose vertices are intersections or dead ends, and whose edges are stretches of streets without intersections. This graph has both undirected edges, which correspond to stretches of two-way streets, and directed edges, which correspond to stretches of one-way streets. Thus, in this way, a graph modeling a city map is a mixed graph.*

Example 14.4: *Physical examples of graphs are present in the electrical wiring and plumbing networks of a building. Such networks can be modeled as graphs, where each connector, fixture, or outlet is viewed as a vertex, and each uninterrupted stretch of wire or pipe is viewed as an edge. Such graphs are actually components of much larger graphs, namely the local power and water distribution networks. Depending on the specific aspects of these graphs that we are interested in, we may consider their edges as undirected or directed, for, in principle, water can flow in a pipe and current can flow in a wire in either direction.*

The two vertices joined by an edge are called the ***end vertices*** (or ***endpoints***) of the edge. If an edge is directed, its first endpoint is its ***origin*** and the other is the ***destination*** of the edge. Two vertices u and v are said to be ***adjacent*** if there is an edge whose end vertices are u and v. An edge is said to be ***incident*** to a vertex if the vertex is one of the edge's endpoints. The ***outgoing edges*** of a vertex are the directed edges whose origin is that vertex. The ***incoming edges*** of a vertex are the directed edges whose destination is that vertex. The ***degree*** of a vertex v, denoted $\deg(v)$, is the number of incident edges of v. The ***in-degree*** and ***out-degree*** of a vertex v are the number of the incoming and outgoing edges of v, and are denoted $\text{indeg}(v)$ and $\text{outdeg}(v)$, respectively.

Example 14.5: *We can study air transportation by constructing a graph G, called a* **flight network**, *whose vertices are associated with airports, and whose edges are associated with flights. (See Figure 14.2.) In graph G, the edges are directed because a given flight has a specific travel direction. The endpoints of an edge e in G correspond respectively to the origin and destination of the flight corresponding to e. Two airports are adjacent in G if there is a flight that flies between them, and an edge e is incident to a vertex v in G if the flight for e flies to or from the airport for v. The outgoing edges of a vertex v correspond to the outbound flights from v's airport, and the incoming edges correspond to the inbound flights to v's airport. Finally, the in-degree of a vertex v of G corresponds to the number of inbound flights to v's airport, and the out-degree of a vertex v in G corresponds to the number of outbound flights.*

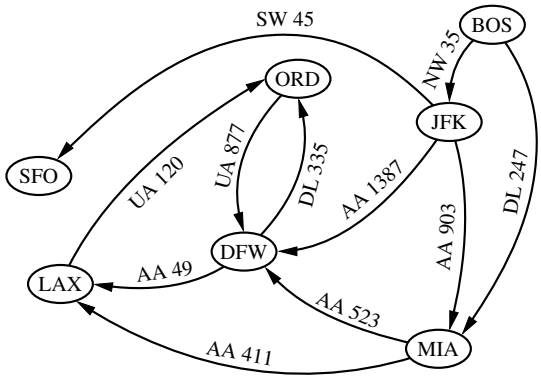


Figure 14.2: Example of a directed graph representing a flight network. The endpoints of edge UA 120 are LAX and ORD; hence, LAX and ORD are adjacent. The in-degree of DFW is 3, and the out-degree of DFW is 2.

The definition of a graph refers to the group of edges as a ***collection***, not a ***set***, thus allowing two undirected edges to have the same end vertices, and for two directed edges to have the same origin and the same destination. Such edges are called ***parallel edges*** or ***multiple edges***. A flight network can contain parallel edges (Example 14.5), such that multiple edges between the same pair of vertices could indicate different flights operating on the same route at different times of the day. Another special type of edge is one that connects a vertex to itself. Namely, we say that an edge (undirected or directed) is a ***self-loop*** if its two endpoints coincide. A self-loop may occur in a graph associated with a city map (Example 14.3), where it would correspond to a "circle" (a curving street that returns to its starting point).

With few exceptions, graphs do not have parallel edges or self-loops. Such graphs are said to be ***simple***. Thus, we can usually say that the edges of a simple graph are a ***set*** of vertex pairs (and not just a collection). Throughout this chapter, we assume that a graph is simple unless otherwise specified.

A ***path*** is a sequence of alternating vertices and edges that starts at a vertex and ends at a vertex such that each edge is incident to its predecessor and successor vertex. A ***cycle*** is a path that starts and ends at the same vertex, and that includes at least one edge. We say that a path is ***simple*** if each vertex in the path is distinct, and we say that a cycle is ***simple*** if each vertex in the cycle is distinct, except for the first and last one. A ***directed path*** is a path such that all edges are directed and are traversed along their direction. A ***directed cycle*** is similarly defined. For example, in Figure 14.2, (BOS, NW 35, JFK, AA 1387, DFW) is a directed simple path, and (LAX, UA 120, ORD, UA 877, DFW, AA 49, LAX) is a directed simple cycle. Note that a directed graph may have a cycle consisting of two edges with opposite direction between the same pair of vertices, for example (ORD, UA 877, DFW, DL 335, ORD) in Figure 14.2. A directed graph is ***acyclic*** if it has no directed cycles. For example, if we were to remove the edge UA 877 from the graph in Figure 14.2, the remaining graph is acyclic. If a graph is simple, we may omit the edges when describing path P or cycle C, as these are well defined, in which case P is a list of adjacent vertices and C is a cycle of adjacent vertices.

Example 14.6: *Given a graph G representing a city map (see Example 14.3), we can model a couple driving to dinner at a recommended restaurant as traversing a path though G. If they know the way, and do not accidentally go through the same intersection twice, then they traverse a simple path in G. Likewise, we can model the entire trip the couple takes, from their home to the restaurant and back, as a cycle. If they go home from the restaurant in a completely different way than how they went, not even going through the same intersection twice, then their entire round trip is a simple cycle. Finally, if they travel along one-way streets for their entire trip, we can model their night out as a directed cycle.*

Given vertices u and v of a (directed) graph G, we say that u ***reaches*** v, and that v is ***reachable*** from u, if G has a (directed) path from u to v. In an undirected graph, the notion of ***reachability*** is symmetric, that is to say, u reaches v if an only if v reaches u. However, in a directed graph, it is possible that u reaches v but v does not reach u, because a directed path must be traversed according to the respective directions of the edges. A graph is ***connected*** if, for any two vertices, there is a path between them. A directed graph $\vec{G}$ is ***strongly connected*** if for any two vertices u and v of $\vec{G}$, u reaches v and v reaches u. (See Figure 14.3 for some examples.)

A ***subgraph*** of a graph G is a graph H whose vertices and edges are subsets of the vertices and edges of G, respectively. A ***spanning subgraph*** of G is a subgraph of G that contains all the vertices of the graph G. If a graph G is not connected, its maximal connected subgraphs are called the ***connected components*** of G. A ***forest*** is a graph without cycles. A ***tree*** is a connected forest, that is, a connected graph without cycles. A ***spanning tree*** of a graph is a spanning subgraph that is a tree. (Note that this definition of a tree is somewhat different from the one given in Chapter 8, as there is not necessarily a designated root.)

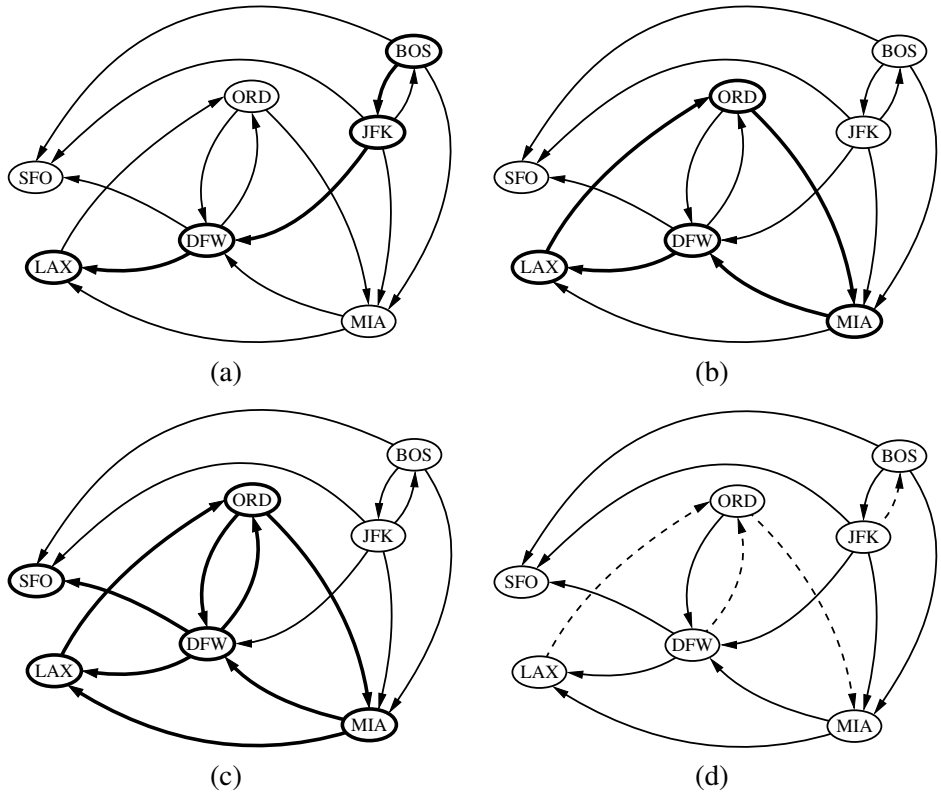


Figure 14.3: Examples of reachability in a directed graph: (a) a directed path from BOS to LAX is highlighted; (b) a directed cycle (ORD, MIA, DFW, LAX, ORD) is highlighted; its vertices induce a strongly connected subgraph; (c) the subgraph of the vertices and edges reachable from ORD is highlighted; (d) the removal of the dashed edges results in an acyclic directed graph.

Example 14.7: *Perhaps the most talked about graph today is the Internet, which can be viewed as a graph whose vertices are computers and whose (undirected) edges are communication connections between pairs of computers on the Internet. The computers and the connections between them in a single domain, like* wiley.com, *form a subgraph of the Internet. If this subgraph is connected, then two users on computers in this domain can send email to one another without having their information packets ever leave their domain. Suppose the edges of this subgraph form a spanning tree. This implies that, if even a single connection goes down (for example, because someone pulls a communication cable out of the back of a computer in this domain), then this subgraph will no longer be connected.*

In the propositions that follow, we explore a few important properties of graphs.

Proposition 14.8: *If G is a graph with m edges and vertex set V, then*

$$\sum_{v \text{ in } V} deg(v) = 2m.$$

Justification: An edge (u,v) is counted twice in the summation above; once by its endpoint u and once by its endpoint v. Thus, the total contribution of the edges to the degrees of the vertices is twice the number of edges. ■

Proposition 14.9: *If G is a directed graph with m edges and vertex set V, then*

$$\sum_{v \text{ in } V} indeg(v) = \sum_{v \text{ in } V} outdeg(v) = m.$$

Justification: In a directed graph, an edge (u,v) contributes one unit to the out-degree of its origin u and one unit to the in-degree of its destination v. Thus, the total contribution of the edges to the out-degrees of the vertices is equal to the number of edges, and similarly for the in-degrees. ■

We next show that a simple graph with n vertices has $O(n^2)$ edges.

Proposition 14.10: *Let G be a simple graph with n vertices and m edges. If G is undirected, then $m \leq n(n-1)/2$, and if G is directed, then $m \leq n(n-1)$.*

Justification: Suppose that G is undirected. Since no two edges can have the same endpoints and there are no self-loops, the maximum degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.8, $2m \leq n(n-1)$. Now suppose that G is directed. Since no two edges can have the same origin and destination, and there are no self-loops, the maximum in-degree of a vertex in G is $n-1$ in this case. Thus, by Proposition 14.9, $m \leq n(n-1)$. ■

There are a number of simple properties of trees, forests, and connected graphs.

Proposition 14.11: *Let G be an undirected graph with n vertices and m edges.*

- *If G is connected, then $m \geq n-1$.*
- *If G is a tree, then $m = n-1$.*
- *If G is a forest, then $m \leq n-1$.*

14.1.1 The Graph ADT

A graph is a collection of vertices and edges. We model the abstraction as a combination of three data types: Vertex, Edge, and Graph. A Vertex is a lightweight object that stores an arbitrary element provided by the user (e.g., an airport code); we assume it supports a method, element( ), to retrieve the stored element. An Edge also stores an associated object (e.g., a flight number, travel distance, cost), retrieved with the element( ) method. In addition, we assume that an Edge supports the following methods:

endpoints( ): Return a tuple (u,v) such that vertex u is the origin of the edge and vertex v is the destination; for an undirected graph, the orientation is arbitrary.

opposite(v): Assuming vertex v is one endpoint of the edge (either origin or destination), return the other endpoint.

The primary abstraction for a graph is the Graph ADT. We presume that a graph can be either ***undirected*** or ***directed***, with the designation declared upon construction; recall that a mixed graph can be represented as a directed graph, modeling edge $\{u,v\}$ as a pair of directed edges (u,v) and (v,u). The Graph ADT includes the following methods:

vertex_count( ): Return the number of vertices of the graph.

vertices( ): Return an iteration of all the vertices of the graph.

edge_count( ): Return the number of edges of the graph.

edges( ): Return an iteration of all the edges of the graph.

get_edge(u,v): Return the edge from vertex u to vertex v, if one exists; otherwise return None. For an undirected graph, there is no difference between get_edge(u,v) and get_edge(v,u).

degree(v, out=True): For an undirected graph, return the number of edges incident to vertex v. For a directed graph, return the number of outgoing (resp. incoming) edges incident to vertex v, as designated by the optional parameter.

incident_edges(v, out=True): Return an iteration of all edges incident to vertex v. In the case of a directed graph, report outgoing edges by default; report incoming edges if the optional parameter is set to False.

insert_vertex(x=None): Create and return a new Vertex storing element x.

insert_edge(u, v, x=None): Create and return a new Edge from vertex u to vertex v, storing element x (None by default).

remove_vertex(v): Remove vertex v and all its incident edges from the graph.

remove_edge(e): Remove edge e from the graph.

14.2 Data Structures for Graphs

In this section, we introduce four data structures for representing a graph. In each representation, we maintain a collection to store the vertices of a graph. However, the four representations differ greatly in the way they organize the edges.

- In an ***edge list***, we maintain an unordered list of all edges. This minimally suffices, but there is no efficient way to locate a particular edge (u,v), or the set of all edges incident to a vertex v.
- In an ***adjacency list***, we maintain, for each vertex, a separate list containing those edges that are incident to the vertex. The complete set of edges can be determined by taking the union of the smaller sets, while the organization allows us to more efficiently find all edges incident to a given vertex.
- An ***adjacency map*** is very similar to an adjacency list, but the secondary container of all edges incident to a vertex is organized as a map, rather than as a list, with the adjacent vertex serving as a key. This allows for access to a specific edge (u,v) in $O(1)$ expected time.
- An ***adjacency matrix*** provides worst-case $O(1)$ access to a specific edge (u,v) by maintaining an $n \times n$ matrix, for a graph with n vertices. Each entry is dedicated to storing a reference to the edge (u,v) for a particular pair of vertices u and v; if no such edge exists, the entry will be None.

A summary of the performance of these structures is given in Table 14.1. We give further explanation of the structures in the remainder of this section.

Operation	Edge List	Adj. List	Adj. Map	Adj. Matrix
vertex_count( )	$O(1)$	$O(1)$	$O(1)$	$O(1)$
edge_count( )	$O(1)$	$O(1)$	$O(1)$	$O(1)$
vertices( )	$O(n)$	$O(n)$	$O(n)$	$O(n)$
edges( )	$O(m)$	$O(m)$	$O(m)$	$O(m)$
get_edge(u,v)	$O(m)$	$O(\min(d_u, d_v))$	$O(1)$ exp.	$O(1)$
degree(v)	$O(m)$	$O(1)$	$O(1)$	$O(n)$
incident_edges(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n)$
insert_vertex(x)	$O(1)$	$O(1)$	$O(1)$	$O(n^2)$
remove_vertex(v)	$O(m)$	$O(d_v)$	$O(d_v)$	$O(n^2)$
insert_edge(u,v,x)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$
remove_edge(e)	$O(1)$	$O(1)$	$O(1)$ exp.	$O(1)$

Table 14.1: A summary of the running times for the methods of the graph ADT, using the graph representations discussed in this section. We let n denote the number of vertices, m the number of edges, and d_v the degree of vertex v. Note that the adjacency matrix uses $O(n^2)$ space, while all other structures use $O(n+m)$ space.

14.2.1 Edge List Structure

The ***edge list*** structure is possibly the simplest, though not the most efficient, representation of a graph G. All vertex objects are stored in an unordered list V, and all edge objects are stored in an unordered list E. We illustrate an example of the edge list structure for a graph G in Figure 14.4.

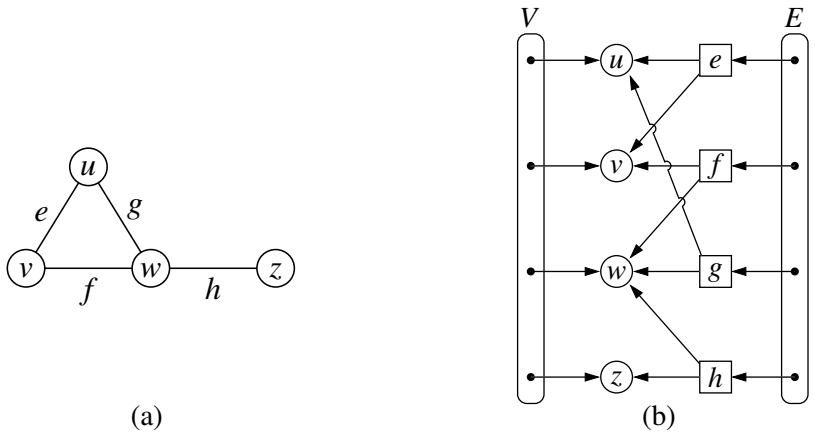


Figure 14.4: (a) A graph G; (b) schematic representation of the edge list structure for G. Notice that an edge object refers to the two vertex objects that correspond to its endpoints, but that vertices do not refer to incident edges.

To support the many methods of the Graph ADT (Section 14.1), we assume the following additional features of an edge list representation. Collections V and E are represented with doubly linked lists using our PositionalList class from Chapter 7.

Vertex Objects

The vertex object for a vertex v storing element x has instance variables for:

- A reference to element x, to support the element( ) method.
- A reference to the position of the vertex instance in the list V, thereby allowing v to be efficiently removed from V if it were removed from the graph.

Edge Objects

The edge object for an edge e storing element x has instance variables for:

- A reference to element x, to support the element( ) method.
- References to the vertex objects associated with the endpoint vertices of e. These allow the edge instance to provide constant-time support for methods endpoints( ) and opposite(v).
- A reference to the position of the edge instance in list E, thereby allowing e to be efficiently removed from E if it were removed from the graph.

Performance of the Edge List Structure

The performance of an edge list structure in fulfilling the graph ADT is summarized in Table 14.2. We begin by discussing the space usage, which is $O(n+m)$ for representing a graph with n vertices and m edges. Each individual vertex or edge instance uses $O(1)$ space, and the additional lists V and E use space proportional to their number of entries.

In terms of running time, the edge list structure does as well as one could hope in terms of reporting the number of vertices or edges, or in producing an iteration of those vertices or edges. By querying the respective list V or E, the vertex_count and edge_count methods run in $O(1)$ time, and by iterating through the appropriate list, the methods vertices and edges run respectively in $O(n)$ and $O(m)$ time.

The most significant limitations of an edge list structure, especially when compared to the other graph representations, are the $O(m)$ running times of methods get_edge(u,v), degree(v), and incident_edges(v). The problem is that with all edges of the graph in an unordered list E, the only way to answer those queries is through an exhaustive inspection of all edges. The other data structures introduced in this section will implement these methods more efficiently.

Finally, we consider the methods that update the graph. It is easy to add a new vertex or a new edge to the graph in $O(1)$ time. For example, a new edge can be added to the graph by creating an Edge instance storing the given element as data, adding that instance to the positional list E, and recording its resulting Position within E as an attribute of the edge. That stored position can later be used to locate and remove this edge from E in $O(1)$ time, and thus implement the method remove_edge(e)

It is worth discussing why the remove_vertex(v) method has a running time of $O(m)$. As stated in the graph ADT, when a vertex v is removed from the graph, all edges incident to v must also be removed (otherwise, we would have a contradiction of edges that refer to vertices that are not part of the graph). To locate the incident edges to the vertex, we must examine all edges of E.

Operation	Running Time
vertex_count( ), edge_count( )	$O(1)$
vertices( )	$O(n)$
edges( )	$O(m)$
get_edge(u,v), degree(v), incident_edges(v)	$O(m)$
insert_vertex(x), insert_edge(u,v,x), remove_edge(e)	$O(1)$
remove_vertex(v)	$O(m)$

Table 14.2: Running times of the methods of a graph implemented with the edge list structure. The space used is $O(n+m)$, where n is the number of vertices and m is the number of edges.

14.2.2 Adjacency List Structure

In contrast to the edge list representation of a graph, the ***adjacency list*** structure groups the edges of a graph by storing them in smaller, secondary containers that are associated with each individual vertex. Specifically, for each vertex v, we maintain a collection $I(v)$, called the ***incidence collection*** of v, whose entries are edges incident to v. (In the case of a directed graph, outgoing and incoming edges can be respectively stored in two separate collections, $I_{\text{out}}(v)$ and $I_{\text{in}}(v)$.) Traditionally, the incidence collection $I(v)$ for a vertex v is a list, which is why we call this way of representing a graph the ***adjacency list*** structure.

We require that the primary structure for an adjacency list maintain the collection V of vertices in a way so that we can locate the secondary structure $I(v)$ for a given vertex v in $O(1)$ time. This could be done by using a positional list to represent V, with each Vertex instance maintaining a direct reference to its $I(v)$ incidence collection; we illustrate such an adjacency list structure of a graph in Figure 14.5. If vertices can be uniquely numbered from 0 to $n-1$, we could instead use a primary array-based structure to access the appropriate secondary lists.

The primary benefit of an adjacency list is that the collection $I(v)$ contains exactly those edges that should be reported by the method incident_edges(v). Therefore, we can implement this method by iterating the edges of $I(v)$ in $O(\deg(v))$ time, where $\deg(v)$ is the degree of vertex v. This is the best possible outcome for any graph representation, because there are $\deg(v)$ edges to be reported.

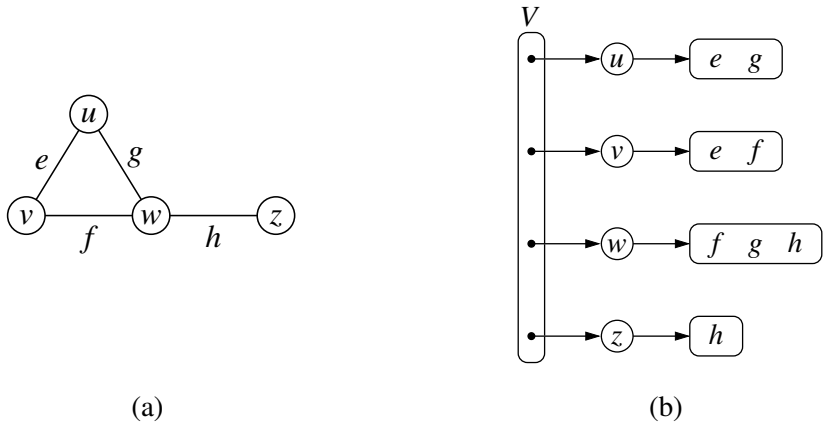


Figure 14.5: (a) An undirected graph G; (b) a schematic representation of the adjacency list structure for G. Collection V is the primary list of vertices, and each vertex has an associated list of incident edges. Although not diagrammed as such, we presume that each edge of the graph is represented with a unique Edge instance that maintains references to its endpoint vertices.

Performance of the Adjacency List Structure

Table 14.3 summarizes the performance of the adjacency list structure implementation of a graph, assuming that the primary collection V and all secondary collections $I(v)$ are implemented with doubly linked lists.

Asymptotically, the space requirements for an adjacency list are the same as an edge list structure, using $O(n+m)$ space for a graph with n vertices and m edges. The primary list of vertices uses $O(n)$ space. The sum of the lengths of all secondary lists is $O(m)$, for reasons that were formalized in Propositions 14.8 and 14.9. In short, an undirected edge (u,v) is referenced in both $I(u)$ and $I(v)$, but its presence in the graph results in only a constant amount of additional space.

We have already noted that the incident_edges(v) method can be achieved in $O(\deg(v))$ time based on use of $I(v)$. We can achieve the degree(v) method of the graph ADT to use $O(1)$ time, assuming collection $I(v)$ can report its size in similar time. To locate a specific edge for implementing get_edge(u,v), we can search through either $I(u)$ and $I(v)$. By choosing the smaller of the two, we get $O(\min(\deg(u),\deg(v)))$ running time.

The rest of the bounds in Table 14.3 can be achieved with additional care. To efficiently support deletions of edges, an edge (u,v) would need to maintain a reference to its positions within both $I(u)$ and $I(v)$, so that it could be deleted from those collections in $O(1)$ time. To remove a vertex v, we must also remove any incident edges, but at least we can locate those edges in $O(\deg(v))$ time.

The easiest way to support edges( ) in $O(m)$ and count_edges( ) in $O(1)$ is to maintain an auxiliary list E of edges, as in the edge list representation. Otherwise, we can implement the edges method in $O(n+m)$ time by accessing each secondary list and reporting its edges, taking care not to report an undirected edge (u,v) twice.

Operation	Running Time
vertex_count( ), edge_count( )	$O(1)$
vertices( )	$O(n)$
edges( )	$O(m)$
get_edge(u,v)	$O(\min(\deg(u),\deg(v)))$
degree(v)	$O(1)$
incident_edges(v)	$O(\deg(v))$
insert_vertex(x), insert_edge(u,v,x)	$O(1)$
remove_edge(e)	$O(1)$
remove_vertex(v)	$O(\deg(v))$

Table 14.3: Running times of the methods of a graph implemented with the adjacency list structure. The space used is $O(n+m)$, where n is the number of vertices and m is the number of edges.

14.2.3 Adjacency Map Structure

In the adjacency list structure, we assume that the secondary incidence collections are implemented as unordered linked lists. Such a collection $I(v)$ uses space proportional to $O(\deg(v))$, allows an edge to be added or removed in $O(1)$ time, and allows an iteration of all edges incident to vertex v in $O(\deg(v))$ time. However, the best implementation of get_edge(u,v) requires $O(\min(\deg(u), \deg(v)))$ time, because we must search through either $I(u)$ or $I(v)$.

We can improve the performance by using a hash-based map to implement $I(v)$ for each vertex v. Specifically, we let the opposite endpoint of each incident edge serve as a key in the map, with the edge structure serving as the value. We call such a graph representation an ***adjacency map***. (See Figure 14.6.) The space usage for an adjacency map remains $O(n+m)$, because $I(v)$ uses $O(\deg(v))$ space for each vertex v, as with the adjacency list.

The advantage of the adjacency map, relative to an adjacency list, is that the get_edge(u,v) method can be implemented in ***expected*** $O(1)$ time by searching for vertex u as a key in $I(v)$, or vice versa. This provides a likely improvement over the adjacency list, while retaining the worst-case bound of $O(\min(\deg(u), \deg(v)))$.

In comparing the performance of adjacency map to other representations (see Table 14.1), we find that it essentially achieves optimal running times for all methods, making it an excellent all-purpose choice as a graph representation.

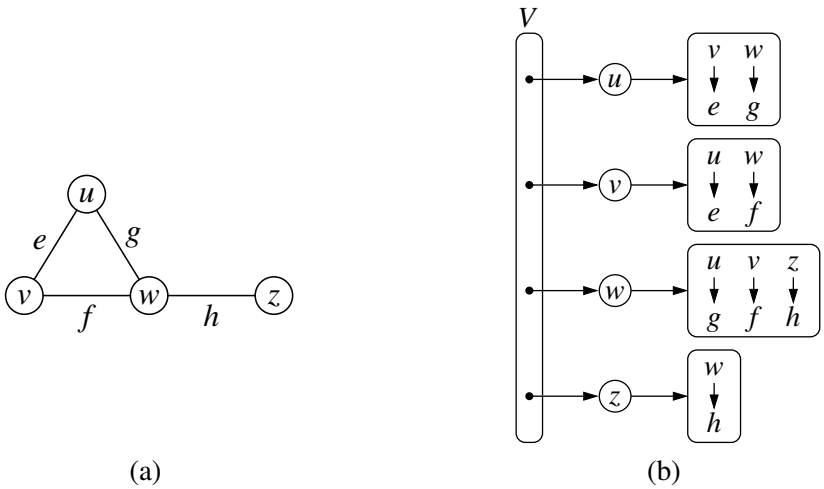


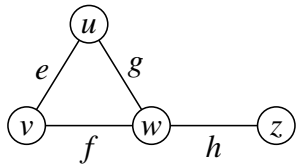
Figure 14.6: (a) An undirected graph G; (b) a schematic representation of the adjacency map structure for G. Each vertex maintains a secondary map in which neighboring vertices serve as keys, with the connecting edges as associated values. Although not diagrammed as such, we presume that there is a unique Edge instance for each edge of the graph, and that it maintains references to its endpoint vertices.

14.2.4 Adjacency Matrix Structure

The ***adjacency matrix*** structure for a graph G augments the edge list structure with a matrix A (that is, a two-dimensional array, as in Section 5.6), which allows us to locate an edge between a given pair of vertices in *worst-case* constant time. In the adjacency matrix representation, we think of the vertices as being the integers in the set $\{0,1,\ldots,n-1\}$ and the edges as being pairs of such integers. This allows us to store references to edges in the cells of a two-dimensional $n \times n$ array A. Specifically, the cell $A[i,j]$ holds a reference to the edge (u,v), if it exists, where u is the vertex with index i and v is the vertex with index j. If there is no such edge, then $A[i,j] = \text{None}$. We note that array A is symmetric if graph G is undirected, as $A[i,j] = A[j,i]$ for all pairs i and j. (See Figure 14.7.)

The most significant advantage of an adjacency matrix is that any edge (u,v) can be accessed in worst-case $O(1)$ time; recall that the adjacency map supports that operation in $O(1)$ *expected* time. However, several operation are less efficient with an adjacency matrix. For example, to find the edges incident to vertex v, we must presumably examine all n entries in the row associated with v; recall that an adjacency list or map can locate those edges in optimal $O(\deg(v))$ time. Adding or removing vertices from a graph is problematic, as the matrix must be resized.

Furthermore, the $O(n^2)$ space usage of an adjacency matrix is typically far worse than the $O(n+m)$ space required of the other representations. Although, in the worst case, the number of edges in a ***dense*** graph will be proportional to n^2, most real-world graphs are ***sparse***. In such cases, use of an adjacency matrix is inefficient. However, if a graph is dense, the constants of proportionality of an adjacency matrix can be smaller than that of an adjacency list or map. In fact, if edges do not have auxiliary data, a Boolean adjacency matrix can use one bit per edge slot, such that $A[i,j] = \text{True}$ if and only if associated (u,v) is an edge.



(a)

		0	1	2	3
u	0		e	g	
v	1	e		f	
w	2	g	f		h
z	3			h	

(b)

Figure 14.7: (a) An undirected graph G; (b) a schematic representation of the auxiliary adjacency matrix structure for G, in which n vertices are mapped to indices 0 to $n-1$. Although not diagrammed as such, we presume that there is a unique Edge instance for each edge, and that it maintains references to its endpoint vertices. We also assume that there is a secondary edge list (not pictured), to allow the edges( ) method to run in $O(m)$ time, for a graph with m edges.

14.2.5 Python Implementation

In this section, we provide an implementation of the Graph ADT. Our implementation will support directed or undirected graphs, but for ease of explanation, we first describe it in the context of an undirected graph.

We use a variant of the ***adjacency map*** representation. For each vertex v, we use a Python dictionary to represent the secondary incidence map $I(v)$. However, we do not explicitly maintain lists V and E, as originally described in the edge list representation. The list V is replaced by a top-level dictionary D that maps each vertex v to its incidence map $I(v)$; note that we can iterate through all vertices by generating the set of keys for dictionary D. By using such a dictionary D to map vertices to the secondary incidence maps, we need not maintain references to those incidence maps as part of the vertex structures. Also, a vertex does not need to explicitly maintain a reference to its position in D, because it can be determined in $O(1)$ expected time. This greatly simplifies our implementation. However, a consequence of our design is that some of the worst-case running time bounds for the graph ADT operations, given in Table 14.1, become ***expected*** bounds. Rather than maintain list E, we are content with taking the union of the edges found in the various incidence maps; technically, this runs in $O(n+m)$ time rather than strictly $O(m)$ time, as the dictionary D has n keys, even if some incidence maps are empty.

Our implementation of the graph ADT is given in Code Fragments 14.1 through 14.3. Classes Vertex and Edge, given in Code Fragment 14.1, are rather simple, and can be nested within the more complex Graph class. Note that we define the __hash__ method for both Vertex and Edge so that those instances can be used as keys in Python's hash-based sets and dictionaries. The rest of the Graph class is given in Code Fragments 14.2 and 14.3. Graphs are undirected by default, but can be declared as directed with an optional parameter to the constructor.

Internally, we manage the directed case by having two different top-level dictionary instances, _outgoing and _incoming, such that _outgoing[v] maps to another dictionary representing $I_{\text{out}}(v)$, and _incoming[v] maps to a representation of $I_{\text{in}}(v)$. In order to unify our treatment of directed and undirected graphs, we continue to use the _outgoing and _incoming identifiers in the undirected case, yet as aliases to the same dictionary. For convenience, we define a utility named is_directed to allow us to distinguish between the two cases.

For methods degree and incident_edges, which each accept an optional parameter to differentiate between the outgoing and incoming orientations, we choose the appropriate map before proceeding. For method insert_vertex, we always initialize _outgoing[v] to an empty dictionary for new vertex v. In the directed case, we independently initialize _incoming[v] as well. For the undirected case, that step is unnecessary as _outgoing and _incoming are aliases. We leave the implementations of methods remove_vertex and remove_edge as exercises (C-14.37 and C-14.38).

```
1   #---------------------- nested Vertex class ----------------------
2   class Vertex:
3     """Lightweight vertex structure for a graph."""
4     __slots__ = '_element'
5
6     def __init__(self, x):
7       """Do not call constructor directly. Use Graph's insert_vertex(x)."""
8       self._element = x
9
10    def element(self):
11      """Return element associated with this vertex."""
12      return self._element
13
14    def __hash__(self):             # will allow vertex to be a map/set key
15      return hash(id(self))
16
17  #---------------------- nested Edge class ----------------------
18  class Edge:
19    """Lightweight edge structure for a graph."""
20    __slots__ = '_origin', '_destination', '_element'
21
22    def __init__(self, u, v, x):
23      """Do not call constructor directly. Use Graph's insert_edge(u,v,x)."""
24      self._origin = u
25      self._destination = v
26      self._element = x
27
28    def endpoints(self):
29      """Return (u,v) tuple for vertices u and v."""
30      return (self._origin, self._destination)
31
32    def opposite(self, v):
33      """Return the vertex that is opposite v on this edge."""
34      return self._destination if v is self._origin else self._origin
35
36    def element(self):
37      """Return element associated with this edge."""
38      return self._element
39
40    def __hash__(self):             # will allow edge to be a map/set key
41      return hash( (self._origin, self._destination) )
```

Code Fragment 14.1: Vertex and Edge classes (to be nested within Graph class).

```
1  class Graph:
2    """Representation of a simple graph using an adjacency map."""
3
4    def __init__(self, directed=False):
5      """Create an empty graph (undirected, by default).
6
7      Graph is directed if optional paramter is set to True.
8      """
9      self._outgoing = { }
10     # only create second map for directed graph; use alias for undirected
11     self._incoming = { } if directed else self._outgoing
12
13   def is_directed(self):
14     """Return True if this is a directed graph; False if undirected.
15
16     Property is based on the original declaration of the graph, not its contents.
17     """
18     return self._incoming is not self._outgoing # directed if maps are distinct
19
20   def vertex_count(self):
21     """Return the number of vertices in the graph."""
22     return len(self._outgoing)
23
24   def vertices(self):
25     """Return an iteration of all vertices of the graph."""
26     return self._outgoing.keys( )
27
28   def edge_count(self):
29     """Return the number of edges in the graph."""
30     total = sum(len(self._outgoing[v]) for v in self._outgoing)
31     # for undirected graphs, make sure not to double-count edges
32     return total if self.is_directed( ) else total // 2
33
34   def edges(self):
35     """Return a set of all edges of the graph."""
36     result = set( )          # avoid double-reporting edges of undirected graph
37     for secondary_map in self._outgoing.values( ):
38       result.update(secondary_map.values( ))      # add edges to resulting set
39     return result
```

Code Fragment 14.2: Graph class definition (continued in Code Fragment 14.3).

```
40    def get_edge(self, u, v):
41      """Return the edge from u to v, or None if not adjacent."""
42      return self._outgoing[u].get(v)          # returns None if v not adjacent
43
44    def degree(self, v, outgoing=True):
45      """Return number of (outgoing) edges incident to vertex v in the graph.
46
47      If graph is directed, optional parameter used to count incoming edges.
48      """
49      adj = self._outgoing if outgoing else self._incoming
50      return len(adj[v])
51
52    def incident_edges(self, v, outgoing=True):
53      """Return all (outgoing) edges incident to vertex v in the graph.
54
55      If graph is directed, optional parameter used to request incoming edges.
56      """
57      adj = self._outgoing if outgoing else self._incoming
58      for edge in adj[v].values( ):
59        yield edge
60
61    def insert_vertex(self, x=None):
62      """Insert and return a new Vertex with element x."""
63      v = self.Vertex(x)
64      self._outgoing[v] = { }
65      if self.is_directed( ):
66        self._incoming[v] = { }          # need distinct map for incoming edges
67      return v
68
69    def insert_edge(self, u, v, x=None):
70      """Insert and return a new Edge from u to v with auxiliary element x."""
71      e = self.Edge(u, v, x)
72      self._outgoing[u][v] = e
73      self._incoming[v][u] = e
```

Code Fragment 14.3: Graph class definition (continued from Code Fragment 14.2). We omit error-checking of parameters for brevity.

14.3 Graph Traversals

Greek mythology tells of an elaborate labyrinth that was built to house the monstrous Minotaur, which was part bull and part man. This labyrinth was so complex that neither beast nor human could escape it. No human, that is, until the Greek hero, Theseus, with the help of the king's daughter, Ariadne, decided to implement a ***graph traversal*** algorithm. Theseus fastened a ball of thread to the door of the labyrinth and unwound it as he traversed the twisting passages in search of the monster. Theseus obviously knew about good algorithm design, for, after finding and defeating the beast, Theseus easily followed the string back out of the labyrinth to the loving arms of Ariadne.

Formally, a ***traversal*** is a systematic procedure for exploring a graph by examining all of its vertices and edges. A traversal is efficient if it visits all the vertices and edges in time proportional to their number, that is, in linear time.

Graph traversal algorithms are key to answering many fundamental questions about graphs involving the notion of ***reachability***, that is, in determining how to travel from one vertex to another while following paths of a graph. Interesting problems that deal with reachability in an undirected graph G include the following:

- Computing a path from vertex u to vertex v, or reporting that no such path exists.
- Given a start vertex s of G, computing, for every vertex v of G, a path with the minimum number of edges between s and v, or reporting that no such path exists.
- Testing whether G is connected.
- Computing a spanning tree of G, if G is connected.
- Computing the connected components of G.
- Computing a cycle in G, or reporting that G has no cycles.

Interesting problems that deal with reachability in a directed graph $\vec{G}$ include the following:

- Computing a directed path from vertex u to vertex v, or reporting that no such path exists.
- Finding all the vertices of $\vec{G}$ that are reachable from a given vertex s.
- Determine whether $\vec{G}$ is acyclic.
- Determine whether $\vec{G}$ is strongly connected.

In the remainder of this section, we present two efficient graph traversal algorithms, called ***depth-first search*** and ***breadth-first search***, respectively.

14.3.1 Depth-First Search

The first traversal algorithm we consider in this section is ***depth-first search*** (DFS). Depth-first search is useful for testing a number of properties of graphs, including whether there is a path from one vertex to another and whether or not a graph is connected.

Depth-first search in a graph G is analogous to wandering in a labyrinth with a string and a can of paint without getting lost. We begin at a specific starting vertex s in G, which we initialize by fixing one end of our string to s and painting s as "visited." The vertex s is now our "current" vertex—call our current vertex u. We then traverse G by considering an (arbitrary) edge (u,v) incident to the current vertex u. If the edge (u,v) leads us to a vertex v that is already visited (that is, painted), we ignore that edge. If, on the other hand, (u,v) leads to an unvisited vertex v, then we unroll our string, and go to v. We then paint v as "visited," and make it the current vertex, repeating the computation above. Eventually, we will get to a "dead end," that is, a current vertex v such that all the edges incident to v lead to vertices already visited. To get out of this impasse, we roll our string back up, backtracking along the edge that brought us to v, going back to a previously visited vertex u. We then make u our current vertex and repeat the computation above for any edges incident to u that we have not yet considered. If all of u's incident edges lead to visited vertices, then we again roll up our string and backtrack to the vertex we came from to get to u, and repeat the procedure at that vertex. Thus, we continue to backtrack along the path that we have traced so far until we find a vertex that has yet unexplored edges, take one such edge, and continue the traversal. The process terminates when our backtracking leads us back to the start vertex s, and there are no more unexplored edges incident to s.

The pseudo-code for a depth-first search traversal starting at a vertex u (see Code Fragment 14.4) follows our analogy with string and paint. We use recursion to implement the string analogy, and we assume that we have a mechanism (the paint analogy) to determine whether a vertex or edge has been previously explored.

```
Algorithm DFS(G,u):          {We assume u has already been marked as visited}
  Input: A graph G and a vertex u of G
  Output: A collection of vertices reachable from u, with their discovery edges
  for each outgoing edge e = (u,v) of u do
    if vertex v has not been visited then
      Mark vertex v as visited (via edge e).
      Recursively call DFS(G,v).
```

Code Fragment 14.4: The DFS algorithm.

Classifying Graph Edges with DFS

An execution of depth-first search can be used to analyze the structure of a graph, based upon the way in which edges are explored during the traversal. The DFS process naturally identifies what is known as the ***depth-first search tree*** rooted at a starting vertex s. Whenever an edge $e = (u,v)$ is used to discover a new vertex v during the DFS algorithm of Code Fragment 14.4, that edge is known as a ***discovery edge*** or ***tree edge***, as oriented from u to v. All other edges that are considered during the execution of DFS are known as ***nontree edges***, which take us to a previously visited vertex. In the case of an undirected graph, we will find that all nontree edges that are explored connect the current vertex to one that is an ancestor of it in the DFS tree. We will call such an edge a ***back edge***. When performing a DFS on a directed graph, there are three possible kinds of nontree edges:

- ***back edges***, which connect a vertex to an ancestor in the DFS tree
- ***forward edges***, which connect a vertex to a descendant in the DFS tree
- ***cross edges***, which connect a vertex to a vertex that is neither its ancestor nor its descendant.

An example application of the DFS algorithm on a directed graph is shown in Figure 14.8, demonstrating each type of nontree edge. An example application of the DFS algorithm on an undirected graph is shown in Figure 14.9.

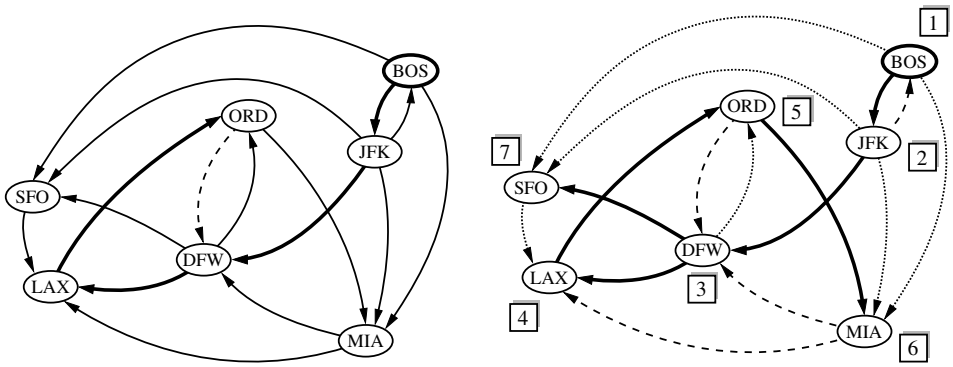


Figure 14.8: An example of a DFS in a directed graph, starting at vertex (BOS): (a) intermediate step, where, for the first time, a considered edge leads to an already visited vertex (DFW); (b) the completed DFS. The tree edges are shown with thick lines, the back edges are shown with dashed lines, and the forward and cross edges are shown with dotted lines. The order in which the vertices are visited is indicated by a label next to each vertex. The edge (ORD,DFW) is a back edge, but (DFW,ORD) is a forward edge. Edge (BOS,SFO) is a forward edge, and (SFO,LAX) is a cross edge.

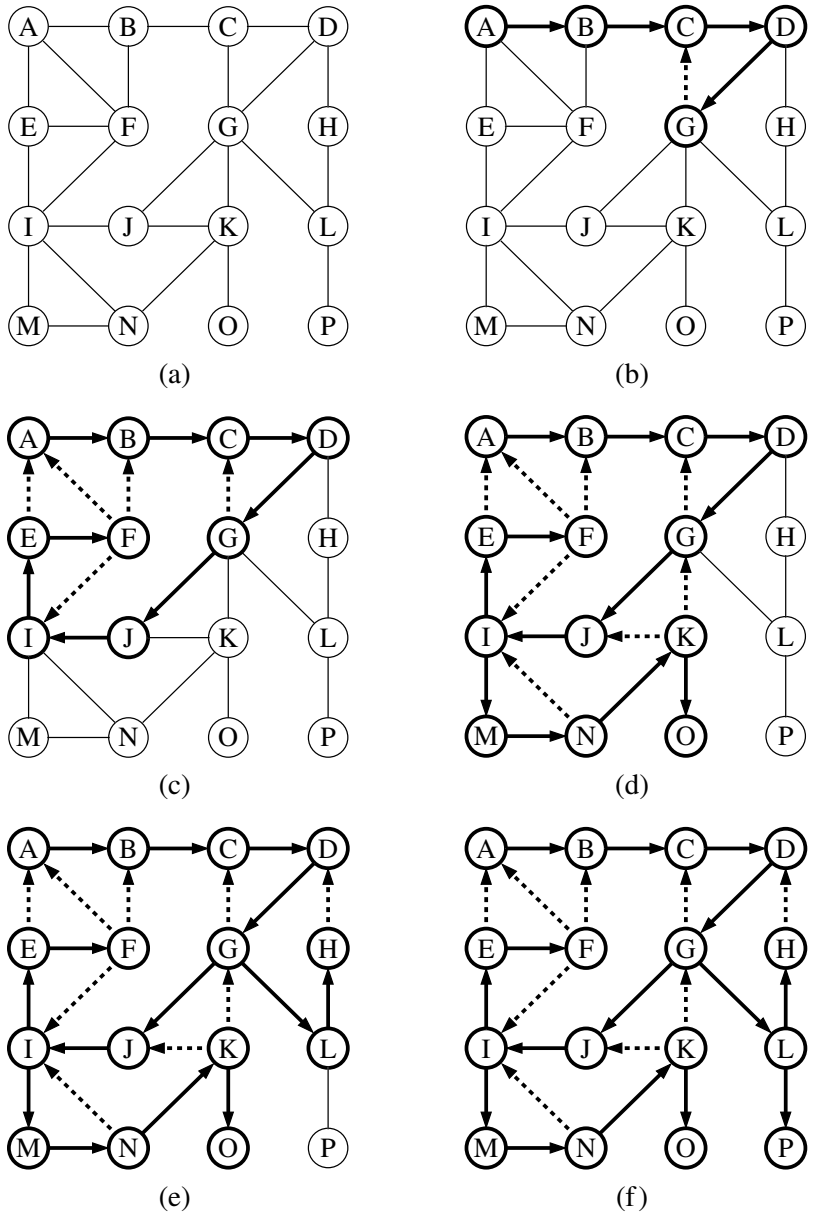


Figure 14.9: Example of depth-first search traversal on an undirected graph starting at vertex A. We assume that a vertex's adjacencies are considered in alphabetical order. Visited vertices and explored edges are highlighted, with discovery edges drawn as solid lines and nontree (back) edges as dashed lines: (a) input graph; (b) path of tree edges, traced from A until back edge (G,C) is examined; (c) reaching F, which is a dead end; (d) after backtracking to I, resuming with edge (I,M), and hitting another dead end at O; (e) after backtracking to G, continuing with edge (G,L), and hitting another dead end at H; (f) final result.

Properties of a Depth-First Search

There are a number of observations that we can make about the depth-first search algorithm, many of which derive from the way the DFS algorithm partitions the edges of a graph G into groups. We begin with the most significant property.

Proposition 14.12: *Let G be an undirected graph on which a DFS traversal starting at a vertex s has been performed. Then the traversal visits all vertices in the connected component of s, and the discovery edges form a spanning tree of the connected component of s.*

Justification: Suppose there is at least one vertex w in s's connected component not visited, and let v be the first unvisited vertex on some path from s to w (we may have $v = w$). Since v is the first unvisited vertex on this path, it has a neighbor u that was visited. But when we visited u, we must have considered the edge (u,v); hence, it cannot be correct that v is unvisited. Therefore, there are no unvisited vertices in s's connected component.

Since we only follow a discovery edge when we go to an unvisited vertex, we will never form a cycle with such edges. Therefore, the discovery edges form a connected subgraph without cycles, hence a tree. Moreover, this is a spanning tree because, as we have just seen, the depth-first search visits each vertex in the connected component of s. ■

Proposition 14.13: *Let $\vec{G}$ be a directed graph. Depth-first search on $\vec{G}$ starting at a vertex s visits all the vertices of $\vec{G}$ that are reachable from s. Also, the DFS tree contains directed paths from s to every vertex reachable from s.*

Justification: Let V_s be the subset of vertices of $\vec{G}$ visited by DFS starting at vertex s. We want to show that V_s contains s and every vertex reachable from s belongs to V_s. Suppose now, for the sake of a contradiction, that there is a vertex w reachable from s that is not in V_s. Consider a directed path from s to w, and let (u,v) be the first edge on such a path taking us out of V_s, that is, u is in V_s but v is not in V_s. When DFS reaches u, it explores all the outgoing edges of u, and thus must reach also vertex v via edge (u,v). Hence, v should be in V_s, and we have obtained a contradiction. Therefore, V_s must contain every vertex reachable from s.

We prove the second fact by induction on the steps of the algorithm. We claim that each time a discovery edge (u,v) is identified, there exists a directed path from s to v in the DFS tree. Since u must have previously been discovered, there exists a path from s to u, so by appending the edge (u,v) to that path, we have a directed path from s to v. ■

Note that since back edges always connect a vertex v to a previously visited vertex u, each back edge implies a cycle in G, consisting of the discovery edges from u to v plus the back edge (u,v).

Running Time of Depth-First Search

In terms of its running time, depth-first search is an efficient method for traversing a graph. Note that DFS is called at most once on each vertex (since it gets marked as visited), and therefore every edge is examined at most twice for an undirected graph, once from each of its end vertices, and at most once in a directed graph, from its origin vertex. If we let $n_s \leq n$ be the number of vertices reachable from a vertex s, and $m_s \leq m$ be the number of incident edges to those vertices, a DFS starting at s runs in $O(n_s + m_s)$ time, provided the following conditions are satisfied:

- The graph is represented by a data structure such that creating and iterating through the incident_edges(v) takes $O(\deg(v))$ time, and the e.opposite(v) method takes $O(1)$ time. The adjacency list structure is one such structure, but the adjacency matrix structure is not.
- We have a way to "mark" a vertex or edge as explored, and to test if a vertex or edge has been explored in $O(1)$ time. We discuss ways of implementing DFS to achieve this goal in the next section.

Given the assumptions above, we can solve a number of interesting problems.

Proposition 14.14: *Let G be an undirected graph with n vertices and m edges. A DFS traversal of G can be performed in $O(n+m)$ time, and can be used to solve the following problems in $O(n+m)$ time:*

- *Computing a path between two given vertices of G, if one exists.*
- *Testing whether G is connected.*
- *Computing a spanning tree of G, if G is connected.*
- *Computing the connected components of G.*
- *Computing a cycle in G, or reporting that G has no cycles.*

Proposition 14.15: *Let $\vec{G}$ be a directed graph with n vertices and m edges. A DFS traversal of $\vec{G}$ can be performed in $O(n+m)$ time, and can be used to solve the following problems in $O(n+m)$ time:*

- *Computing a directed path between two given vertices of $\vec{G}$, if one exists.*
- *Computing the set of vertices of $\vec{G}$ that are reachable from a given vertex s.*
- *Testing whether $\vec{G}$ is strongly connected.*
- *Computing a directed cycle in $\vec{G}$, or reporting that $\vec{G}$ is acyclic.*
- *Computing the* **transitive closure** *of $\vec{G}$ (see Section 14.4).*

The justification of Propositions 14.14 and 14.15 is based on algorithms that use slightly modified versions of the DFS algorithm as subroutines. We will explore some of those extensions in the remainder of this section.

14.3.2 DFS Implementation and Extensions

We begin by providing a Python implementation of the basic depth-first search algorithm, originally described with pseudo-code in Code Fragment 14.4. Our DFS function is presented in Code Fragment 14.5.

```
1  def DFS(g, u, discovered):
2    """Perform DFS of the undiscovered portion of Graph g starting at Vertex u.
3
4    discovered is a dictionary mapping each vertex to the edge that was used to
5    discover it during the DFS. (u should be "discovered" prior to the call.)
6    Newly discovered vertices will be added to the dictionary as a result.
7    """
8    for e in g.incident_edges(u):          # for every outgoing edge from u
9      v = e.opposite(u)
10     if v not in discovered:              # v is an unvisited vertex
11       discovered[v] = e                  # e is the tree edge that discovered v
12       DFS(g, v, discovered)              # recursively explore from v
```

Code Fragment 14.5: Recursive implementation of depth-first search on a graph, starting at a designated vertex u.

In order to track which vertices have been visited, and to build a representation of the resulting DFS tree, our implementation introduces a third parameter, named discovered. This parameter should be a Python dictionary that maps a vertex of the graph to the tree edge that was used to discover that vertex. As a technicality, we assume that the source vertex u occurs as a key of the dictionary, with None as its value. Thus, a caller might start the traversal as follows:

```
result = {u : None}          # a new dictionary, with u trivially discovered
DFS(g, u, result)
```

The dictionary serves two purposes. Internally, the dictionary provides a mechanism for recognizing visited vertices, as they will appear as keys in the dictionary. Externally, the DFS function augments this dictionary as it proceeds, and thus the values within the dictionary are the DFS tree edges at the conclusion of the process.

Because the dictionary is hash-based, the test, "**if** v **not in** discovered," and the record-keeping step, "discovered[v] = e," run in $O(1)$ ***expected*** time, rather than worst-case time. In practice, this is a compromise we are willing to accept, but it does violate the formal analysis of the algorithm, as given on page 643. If we could assume that vertices could be numbered from 0 to $n-1$, then those numbers could be used as indices into an array-based lookup table rather than a hash-based map. Alternatively, we could store each vertex's discovery status and associated tree edge directly as part of the vertex instance.

Reconstructing a Path from u to v

We can use the basic DFS function as a tool to identify the (directed) path leading from vertex u to v, if v is reachable from u. This path can easily be reconstructed from the information that was recorded in the discovery dictionary during the traversal. Code Fragment 14.6 provides an implementation of a secondary function that produces an ordered list of vertices on the path from u to v.

To reconstruct the path, we begin at the *end* of the path, examining the discovery dictionary to determine what edge was used to reach vertex v, and then what the other endpoint of that edge is. We add that vertex to a list, and then repeat the process to determine what edge was used to discover it. Once we have traced the path all the way back to the starting vertex u, we can reverse the list so that it is properly oriented from u to v, and return it to the caller. This process takes time proportional to the length of the path, and therefore it runs in $O(n)$ time (in addition to the time originally spent calling DFS).

```
1  def construct_path(u, v, discovered):
2    path = [ ]                                  # empty path by default
3    if v in discovered:
4      # we build list from v to u and then reverse it at the end
5      path.append(v)
6      walk = v
7      while walk is not u:
8        e = discovered[walk]                    # find edge leading to walk
9        parent = e.opposite(walk)
10       path.append(parent)
11       walk = parent
12     path.reverse( )                           # reorient path from u to v
13   return path
```

Code Fragment 14.6: Function to reconstruct a directed path from u to v, given the trace of discovery from a DFS started at u. The function returns an ordered list of vertices on the path.

Testing for Connectivity

We can use the basic DFS function to determine whether a graph is connected. In the case of an undirected graph, we simply start a depth-first search at an arbitrary vertex and then test whether len(discovered) equals n at the conclusion. If the graph is connected, then by Proposition 14.12, all vertices will have been discovered; conversely, if the graph is not connected, there must be at least one vertex v that is not reachable from u, and that will not be discovered.

For directed graph, $\vec{G}$, we may wish to test whether it is ***strongly connected***, that is, whether for every pair of vertices u and v, both u reaches v and v reaches u. If we start an independent call to DFS from each vertex, we could determine whether this was the case, but those n calls when combined would run in $O(n(n+m))$. However, we can determine if $\vec{G}$ is strongly connected much faster than this, requiring only two depth-first searches.

We begin by performing a depth-first search of our directed graph $\vec{G}$ starting at an arbitrary vertex s. If there is any vertex of $\vec{G}$ that is not visited by this traversal, and is not reachable from s, then the graph is not strongly connected. If this first depth-first search visits each vertex of $\vec{G}$, we need to then check whether s is reachable from all other vertices. Conceptually, we can accomplish this by making a copy of graph $\vec{G}$, but with the orientation of all edges reversed. A depth-first search starting at s in the reversed graph will reach every vertex that could reach s in the original. In practice, a better approach than making a new graph is to reimplement a version of the DFS method that loops through all ***incoming*** edges to the current vertex, rather than all ***outgoing*** edges. Since this algorithm makes just two DFS traversals of $\vec{G}$, it runs in $O(n+m)$ time.

Computing all Connected Components

When a graph is not connected, the next goal we may have is to identify all of the ***connected components*** of an undirected graph, or the ***strongly connected components*** of a directed graph. We begin by discussing the undirected case.

If an initial call to DFS fails to reach all vertices of a graph, we can restart a new call to DFS at one of those unvisited vertices. An implementation of such a comprehensive DFS_all method is given in Code Fragment 14.7.

```
1  def DFS_complete(g):
2    """Perform DFS for entire graph and return forest as a dictionary.
3
4    Result maps each vertex v to the edge that was used to discover it.
5    (Vertices that are roots of a DFS tree are mapped to None.)
6    """
7    forest = { }
8    for u in g.vertices( ):
9      if u not in forest:
10       forest[u] = None                # u will be the root of a tree
11       DFS(g, u, forest)
12   return forest
```

Code Fragment 14.7: Top-level function that returns a DFS forest for an entire graph.

Although the DFS_complete function makes multiple calls to the original DFS function, the total time spent by a call to DFS_complete is $O(n+m)$. For an undirected graph, recall from our original analysis on page 643 that a single call to DFS starting at vertex s runs in time $O(n_s+m_s)$ where n_s is the number of vertices reachable from s, and m_s is the number of incident edges to those vertices. Because each call to DFS explores a different component, the sum of n_s+m_s terms is $n+m$. The $O(n+m)$ total bound applies to the directed case as well, even though the sets of reachable vertices are not necessarily disjoint. However, because the same discovery dictionary is passed as a parameter to all DFS calls, we know that the DFS subroutine is called once on each vertex, and then each outgoing edge is explored only once during the process.

The DFS_complete function can be used to analyze the connected components of an undirected graph. The discovery dictionary it returns represents a ***DFS forest*** for the entire graph. We say this is a forest rather than a tree, because the graph may not be connected. The number of connected components can be determined by the number of vertices in the discovery dictionary that have None as their discovery edge (those are roots of DFS trees). A minor modification to the core DFS method could be used to tag each vertex with a component number when it is discovered. (See Exercise C-14.44.)

The situation is more complex for finding strongly connected components of a directed graph. There exists an approach for computing those components in $O(n+m)$ time, making use of two separate depth-first search traversals, but the details are beyond the scope of this book.

Detecting Cycles with DFS

For both undirected and directed graphs, a cycle exists if and only if a ***back edge*** exists relative to the DFS traversal of that graph. It is easy to see that if a back edge exists, a cycle exists by taking the back edge from the descendant to its ancestor and then following the tree edges back to the descendant. Conversely, if a cycle exists in the graph, there must be a back edge relative to a DFS (although we do not prove this fact here).

Algorithmically, detecting a back edge in the undirected case is easy, because all edges are either tree edges or back edges. In the case of a directed graph, additional modifications to the core DFS implementation are needed to properly categorize a nontree edge as a back edge. When a directed edge is explored leading to a previously visited vertex, we must recognize whether that vertex is an ancestor of the current vertex. This requires some additional bookkeeping, for example, by tagging vertices upon which a recursive call to DFS is still active. We leave details as an exercise (C-14.43).

14.3.3 Breadth-First Search

The advancing and backtracking of a depth-first search, as described in the previous section, defines a traversal that could be physically traced by a single person exploring a graph. In this section, we consider another algorithm for traversing a connected component of a graph, known as a ***breadth-first search*** (BFS). The BFS algorithm is more akin to sending out, in all directions, many explorers who collectively traverse a graph in coordinated fashion.

A BFS proceeds in rounds and subdivides the vertices into ***levels***. BFS starts at vertex s, which is at level 0. In the first round, we paint as "visited," all vertices adjacent to the start vertex s—these vertices are one step away from the beginning and are placed into level 1. In the second round, we allow all explorers to go two steps (i.e., edges) away from the starting vertex. These new vertices, which are adjacent to level 1 vertices and not previously assigned to a level, are placed into level 2 and marked as "visited." This process continues in similar fashion, terminating when no new vertices are found in a level.

A Python implementation of BFS is given in Code Fragment 14.8. We follow a convention similar to that of DFS (Code Fragment 14.5), using a discovered dictionary both to recognize visited vertices, and to record the discovery edges of the BFS tree. We illustrate a BFS traversal in Figure 14.10.

```
1  def BFS(g, s, discovered):
2    """Perform BFS of the undiscovered portion of Graph g starting at Vertex s.
3
4    discovered is a dictionary mapping each vertex to the edge that was used to
5    discover it during the BFS (s should be mapped to None prior to the call).
6    Newly discovered vertices will be added to the dictionary as a result.
7    """
8    level = [s]                          # first level includes only s
9    while len(level) > 0:
10     next_level = [ ]                   # prepare to gather newly found vertices
11     for u in level:
12       for e in g.incident_edges(u):    # for every outgoing edge from u
13         v = e.opposite(u)
14         if v not in discovered:        # v is an unvisited vertex
15           discovered[v] = e            # e is the tree edge that discovered v
16           next_level.append(v)         # v will be further considered in next pass
17     level = next_level                 # relabel 'next' level to become current
```

Code Fragment 14.8: Implementation of breadth-first search on a graph, starting at a designated vertex s.

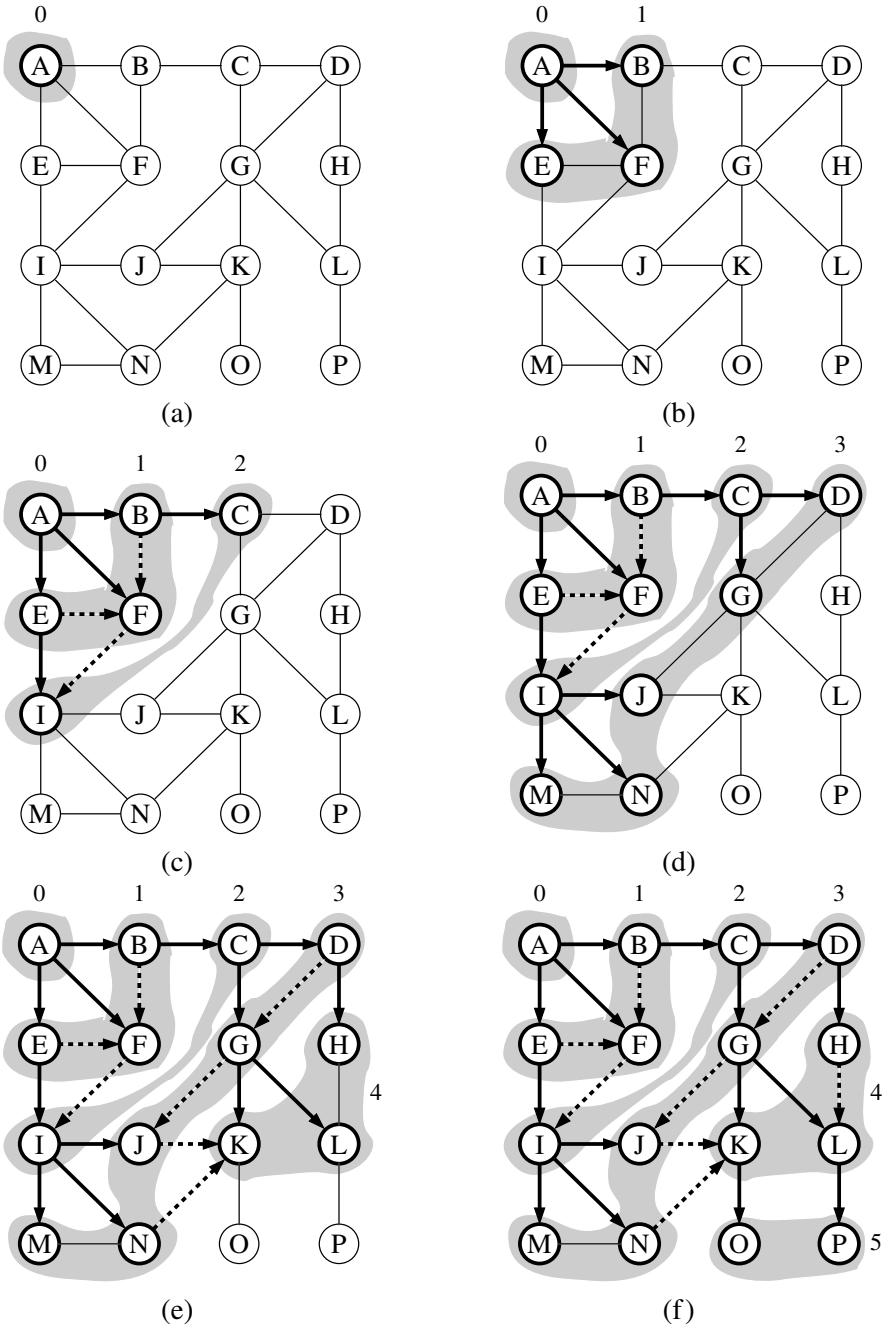


Figure 14.10: Example of breadth-first search traversal, where the edges incident to a vertex are considered in alphabetical order of the adjacent vertices. The discovery edges are shown with solid lines and the nontree (cross) edges are shown with dashed lines: (a) starting the search at A; (b) discovery of level 1; (c) discovery of level 2; (d) discovery of level 3; (e) discovery of level 4; (f) discovery of level 5.

When discussing DFS, we described a classification of nontree edges being either ***back edges***, which connect a vertex to one of its ancestors, ***forward edges***, which connect a vertex to one of its descendants, or ***cross edges***, which connect a vertex to another vertex that is neither its ancestor nor its descendant. For BFS on an undirected graph, all nontree edges are cross edges (see Exercise C-14.47), and for BFS on a directed graph, all nontree edges are either back edges or cross edges (see Exercise C-14.48).

The BFS traversal algorithm has a number of interesting properties, some of which we explore in the proposition that follows. Most notably, a path in a breadth-first search tree rooted at vertex s to any other vertex v is guaranteed to be the shortest such path from s to v in terms of the number of edges.

Proposition 14.16: *Let G be an undirected or directed graph on which a BFS traversal starting at vertex s has been performed. Then*

- *The traversal visits all vertices of G that are reachable from s.*
- *For each vertex v at level i, the path of the BFS tree T between s and v has i edges, and any other path of G from s to v has at least i edges.*
- *If (u,v) is an edge that is not in the BFS tree, then the level number of v can be at most 1 greater than the level number of u.*

We leave the justification of this proposition as an exercise (C-14.50).

The analysis of the running time of BFS is similar to the one of DFS, with the algorithm running in $O(n+m)$ time, or more specifically, in $O(n_s+m_s)$ time if n_s is the number of vertices reachable from vertex s, and $m_s \leq m$ is the number of incident edges to those vertices. To explore the entire graph, the process can be restarted at another vertex, akin to the DFS_complete function of Code Fragment 14.7. Also, the actual path from vertex s to vertex v can be reconstructed using the construct_path function of Code Fragment 14.6

Proposition 14.17: *Let G be a graph with n vertices and m edges represented with the adjacency list structure. A BFS traversal of G takes $O(n+m)$ time.*

Although our implementation of BFS in Code Fragment 14.8 progresses level by level, the BFS algorithm can also be implemented using a single FIFO queue to represent the current fringe of the search. Starting with the source vertex in the queue, we repeatedly remove the vertex from the front of the queue and insert any of its unvisited neighbors to the back of the queue. (See Exercise C-14.51.)

In comparing the capabilities of DFS and BFS, both can be used to efficiently find the set of vertices that are reachable from a given source, and to determine paths to those vertices. However, BFS guarantees that those paths use as few edges as possible. For an undirected graph, both algorithms can be used to test connectivity, to identify connected components, or to locate a cycle. For directed graphs, the DFS algorithm is better suited for certain tasks, such as finding a directed cycle in the graph, or in identifying the strongly connected components.

14.4 Transitive Closure

We have seen that graph traversals can be used to answer basic questions of reachability in a directed graph. In particular, if we are interested in knowing whether there is a path from vertex u to vertex v in a graph, we can perform a DFS or BFS traversal starting at u and observe whether v is discovered. If representing a graph with an adjacency list or adjacency map, we can answer the question of reachability for u and v in $O(n+m)$ time (see Propositions 14.15 and 14.17).

In certain applications, we may wish to answer many reachability queries more efficiently, in which case it may be worthwhile to precompute a more convenient representation of a graph. For example, the first step for a service that computes driving directions from an origin to a destination might be to assess whether the destination is reachable. Similarly, in an electricity network, we may wish to be able to quickly determine whether current flows from one particular vertex to another. Motivated by such applications, we introduce the following definition. The ***transitive closure*** of a directed graph $\vec{G}$ is itself a directed graph $\vec{G}^*$ such that the vertices of $\vec{G}^*$ are the same as the vertices of $\vec{G}$, and $\vec{G}^*$ has an edge (u,v), whenever $\vec{G}$ has a directed path from u to v (including the case where (u,v) is an edge of the original $\vec{G}$).

If a graph is represented as an adjacency list or adjacency map, we can compute its transitive closure in $O(n(n+m))$ time by making use of n graph traversals, one from each starting vertex. For example, a DFS starting at vertex u can be used to determine all vertices reachable from u, and thus a collection of edges originating with u in the transitive closure.

In the remainder of this section, we explore an alternative technique for computing the transitive closure of a directed graph that is particularly well suited for when a directed graph is represented by a data structure that supports $O(1)$-time lookup for the `get_edge(u,v)` method (for example, the adjacency-matrix structure). Let $\vec{G}$ be a directed graph with n vertices and m edges. We compute the transitive closure of $\vec{G}$ in a series of rounds. We initialize $\vec{G}_0 = \vec{G}$. We also arbitrarily number the vertices of $\vec{G}$ as $v_1, v_2, \ldots, v_n$. We then begin the computation of the rounds, beginning with round 1. In a generic round k, we construct directed graph $\vec{G}_k$ starting with $\vec{G}_k = \vec{G}_{k-1}$ and adding to $\vec{G}_k$ the directed edge (v_i, v_j) if directed graph $\vec{G}_{k-1}$ contains both the edges (v_i, v_k) and (v_k, v_j). In this way, we will enforce a simple rule embodied in the proposition that follows.

Proposition 14.18: *For $i = 1, \ldots, n$, directed graph $\vec{G}_k$ has an edge (v_i, v_j) if and only if directed graph $\vec{G}$ has a directed path from v_i to v_j, whose intermediate vertices (if any) are in the set $\{v_1, \ldots, v_k\}$. In particular, $\vec{G}_n$ is equal to $\vec{G}^*$, the transitive closure of $\vec{G}$.*

Proposition 14.18 suggests a simple algorithm for computing the transitive closure of $\vec{G}$ that is based on the series of rounds to compute each $\vec{G}_k$. This algorithm is known as the ***Floyd-Warshall algorithm***, and its pseudo-code is given in Code Fragment 14.9. We illustrate an example run of the Floyd-Warshall algorithm in Figure 14.11.

Algorithm FloydWarshall($\vec{G}$):
Input: A directed graph $\vec{G}$ with n vertices
Output: The transitive closure $\vec{G}^*$ of $\vec{G}$
let $v_1, v_2, \ldots, v_n$ be an arbitrary numbering of the vertices of $\vec{G}$
$\vec{G}_0 = \vec{G}$
for $k = 1$ to n **do**
  $\vec{G}_k = \vec{G}_{k-1}$
  for all i, j in $\{1, \ldots, n\}$ with $i \neq j$ and $i, j \neq k$ **do**
    if both edges (v_i, v_k) and (v_k, v_j) are in $\vec{G}_{k-1}$ **then**
      add edge (v_i, v_j) to $\vec{G}_k$ (if it is not already present)
return $\vec{G}_n$

Code Fragment 14.9: Pseudo-code for the Floyd-Warshall algorithm. This algorithm computes the transitive closure $\vec{G}^*$ of G by incrementally computing a series of directed graphs $\vec{G}_0, \vec{G}_1, \ldots, \vec{G}_n$, for $k = 1, \ldots, n$.

From this pseudo-code, we can easily analyze the running time of the Floyd-Warshall algorithm assuming that the data structure representing G supports methods get_edge and insert_edge in $O(1)$ time. The main loop is executed n times and the inner loop considers each of $O(n^2)$ pairs of vertices, performing a constant-time computation for each one. Thus, the total running time of the Floyd-Warshall algorithm is $O(n^3)$. From the description and analysis above we may immediately derive the following proposition.

Proposition 14.19: *Let $\vec{G}$ be a directed graph with n vertices, and let $\vec{G}$ be represented by a data structure that supports lookup and update of adjacency information in $O(1)$ time. Then the Floyd-Warshall algorithm computes the transitive closure $\vec{G}^*$ of $\vec{G}$ in $O(n^3)$ time.*

Performance of the Floyd-Warshall Algorithm

Asymptotically, the $O(n^3)$ running time of the Floyd-Warshall algorithm is no better than that achieved by repeatedly running DFS, once from each vertex, to compute the reachability. However, the Floyd-Warshall algorithm matches the asymptotic bounds of the repeated DFS when a graph is dense, or when a graph is sparse but represented as an adjacency matrix. (See Exercise R-14.12.)

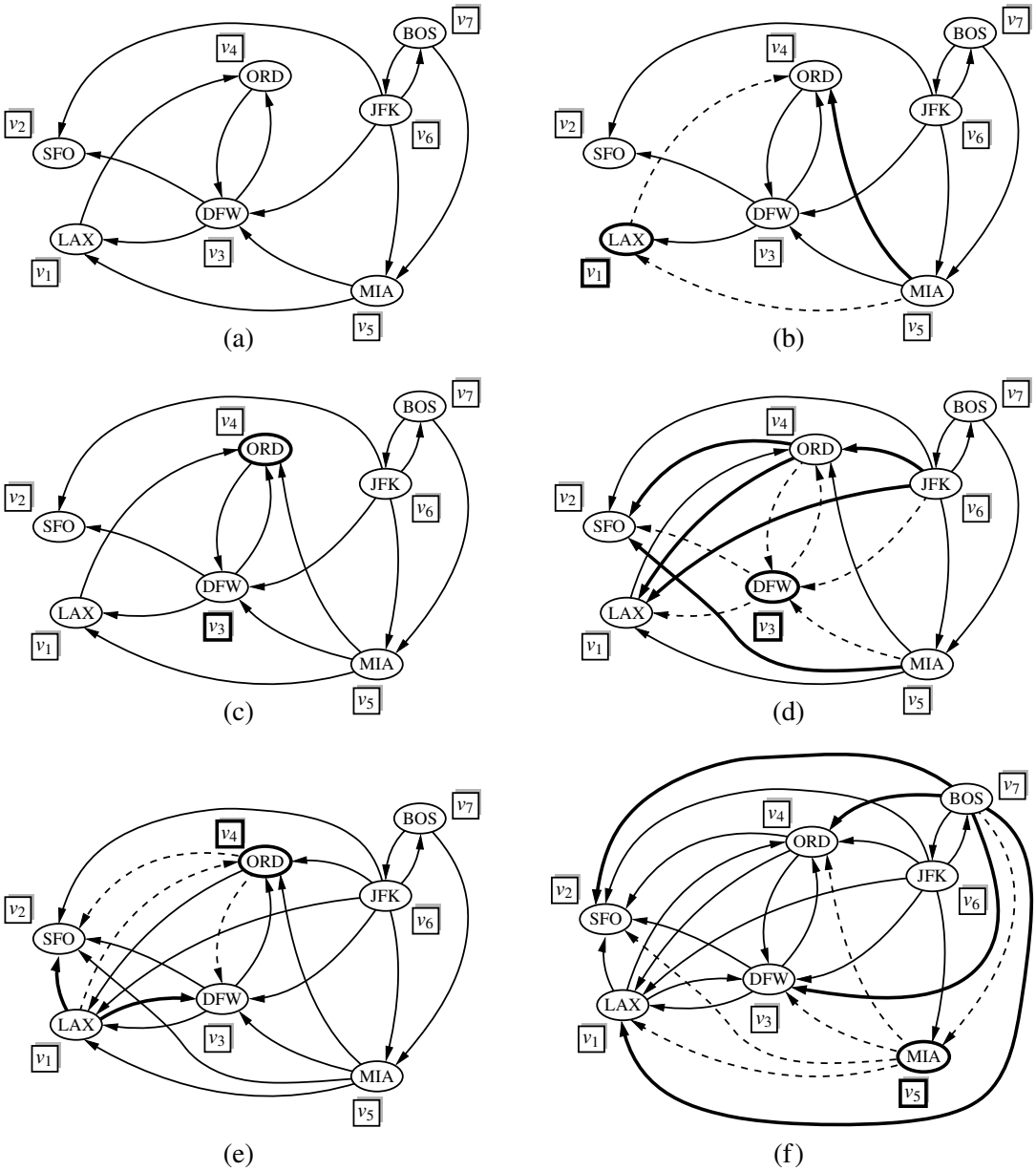


Figure 14.11: Sequence of directed graphs computed by the Floyd-Warshall algorithm: (a) initial directed graph $\vec{G} = \vec{G}_0$ and numbering of the vertices; (b) directed graph $\vec{G}_1$; (c) $\vec{G}_2$; (d) $\vec{G}_3$; (e) $\vec{G}_4$; (f) $\vec{G}_5$. Note that $\vec{G}_5 = \vec{G}_6 = \vec{G}_7$. If directed graph $\vec{G}_{k-1}$ has the edges (v_i, v_k) and (v_k, v_j), but not the edge (v_i, v_j), in the drawing of directed graph $\vec{G}_k$, we show edges (v_i, v_k) and (v_k, v_j) with dashed lines, and edge (v_i, v_j) with a thick line. For example, in (b) existing edges (MIA,LAX) and (LAX,ORD) result in new edge (MIA,ORD).

The importance of the Floyd-Warshall algorithm is that it is much easier to implement than DFS, and much faster in practice because there are relatively few low-level operations hidden within the asymptotic notation. The algorithm is particularly well suited for the use of an adjacency matrix, as a single bit can be used to designate the reachability modeled as an edge (u,v) in the transitive closure.

However, note that repeated calls to DFS results in better asymptotic performance when the graph is sparse and represented using an adjacency list or adjacency map. In that case, a single DFS runs in $O(n+m)$ time, and so the transitive closure can be computed in $O(n^2+nm)$ time, which is preferable to $O(n^3)$.

Python Implementation

We conclude with a Python implementation of the Floyd-Warshall algorithm, as presented in Code Fragment 14.10. Although the original algorithm is described using a series of directed graphs $\vec{G}_0, \vec{G}_1, \ldots, \vec{G}_n$, we create a single copy of the original graph (using the deepcopy method of Python's copy module) and then repeatedly add new edges to the closure as we progress through rounds of the Floyd-Warshall algorithm.

The algorithm requires a canonical numbering of the graph's vertices; therefore, we create a list of the vertices in the closure graph, and subsequently index that list for our order. Within the outermost loop, we must consider all pairs i and j. Finally, we optimize by only iterating through all values of j after we have verified that i has been chosen such that (v_i, v_k) exists in the current version of our closure.

```
1  def floyd_warshall(g):
2    """Return a new graph that is the transitive closure of g."""
3    closure = deepcopy(g)                    # imported from copy module
4    verts = list(closure.vertices( ))        # make indexable list
5    n = len(verts)
6    for k in range(n):
7      for i in range(n):
8        # verify that edge (i,k) exists in the partial closure
9        if i != k and closure.get_edge(verts[i],verts[k]) is not None:
10         for j in range(n):
11           # verify that edge (k,j) exists in the partial closure
12           if i != j != k and closure.get_edge(verts[k],verts[j]) is not None:
13             # if (i,j) not yet included, add it to the closure
14             if closure.get_edge(verts[i],verts[j]) is None:
15               closure.insert_edge(verts[i],verts[j])
16   return closure
```

Code Fragment 14.10: Python implementation of the Floyd-Warshall algorithm.

14.5 Directed Acyclic Graphs

Directed graphs without directed cycles are encountered in many applications. Such a directed graph is often referred to as a ***directed acyclic graph***, or ***DAG***, for short. Applications of such graphs include the following:

- Prerequisites between courses of a degree program.
- Inheritance between classes of an object-oriented program.
- Scheduling constraints between the tasks of a project.

We explore this latter application further in the following example:

Example 14.20: *In order to manage a large project, it is convenient to break it up into a collection of smaller tasks. The tasks, however, are rarely independent, because scheduling constraints exist between them. (For example, in a house building project, the task of ordering nails obviously precedes the task of nailing shingles to the roof deck.) Clearly, scheduling constraints cannot have circularities, because they would make the project impossible. (For example, in order to get a job you need to have work experience, but in order to get work experience you need to have a job.) The scheduling constraints impose restrictions on the order in which the tasks can be executed. Namely, if a constraint says that task a must be completed before task b is started, then a must precede b in the order of execution of the tasks. Thus, if we model a feasible set of tasks as vertices of a directed graph, and we place a directed edge from u to v whenever the task for u must be executed before the task for v, then we define a directed acyclic graph.*

14.5.1 Topological Ordering

The example above motivates the following definition. Let $\vec{G}$ be a directed graph with n vertices. A ***topological ordering*** of $\vec{G}$ is an ordering $v_1,\ldots,v_n$ of the vertices of $\vec{G}$ such that for every edge (v_i,v_j) of $\vec{G}$, it is the case that $i<j$. That is, a topological ordering is an ordering such that any directed path in $\vec{G}$ traverses vertices in increasing order. Note that a directed graph may have more than one topological ordering. (See Figure 14.12.)

Proposition 14.21: *$\vec{G}$ has a topological ordering if and only if it is acyclic.*

Justification: The necessity (the "only if" part of the statement) is easy to demonstrate. Suppose $\vec{G}$ is topologically ordered. Assume, for the sake of a contradiction, that $\vec{G}$ has a cycle consisting of edges $(v_{i_0},v_{i_1}),(v_{i_1},v_{i_2}),\ldots,(v_{i_{k-1}},v_{i_0})$. Because of the topological ordering, we must have $i_0<i_1<\cdots<i_{k-1}<i_0$, which is clearly impossible. Thus, $\vec{G}$ must be acyclic.

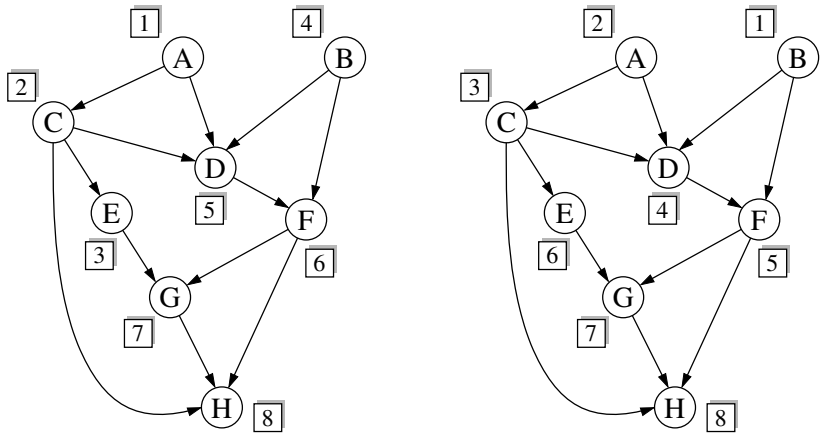


Figure 14.12: Two topological orderings of the same acyclic directed graph.

We now argue the sufficiency of the condition (the "if" part). Suppose $\vec{G}$ is acyclic. We will give an algorithmic description of how to build a topological ordering for $\vec{G}$. Since $\vec{G}$ is acyclic, $\vec{G}$ must have a vertex with no incoming edges (that is, with in-degree 0). Let v_1 be such a vertex. Indeed, if v_1 did not exist, then in tracing a directed path from an arbitrary start vertex, we would eventually encounter a previously visited vertex, thus contradicting the acyclicity of $\vec{G}$. If we remove v_1 from $\vec{G}$, together with its outgoing edges, the resulting directed graph is still acyclic. Hence, the resulting directed graph also has a vertex with no incoming edges, and we let v_2 be such a vertex. By repeating this process until the directed graph becomes empty, we obtain an ordering $v_1, \ldots, v_n$ of the vertices of $\vec{G}$. Because of the construction above, if (v_i, v_j) is an edge of $\vec{G}$, then v_i must be deleted before v_j can be deleted, and thus, $i < j$. Therefore, $v_1, \ldots, v_n$ is a topological ordering. ■

Proposition 14.21's justification suggests an algorithm for computing a topological ordering of a directed graph, which we call ***topological sorting***. We present a Python implementation of the technique in Code Fragment 14.11, and an example execution of the algorithm in Figure 14.13. Our implementation uses a dictionary, named incount, to map each vertex v to a counter that represents the current number of incoming edges to v, excluding those coming from vertices that have previously been added to the topological order. Technically, a Python dictionary provides $O(1)$ expected time access to entries, rather than worst-case time; as was the case with our graph traversals, this could be converted to worst-case time if vertices could be indexed from 0 to $n-1$, or if we store the counter as an element of a vertex.

As a side effect, the topological sorting algorithm of Code Fragment 14.11 also tests whether the given directed graph $\vec{G}$ is acyclic. Indeed, if the algorithm terminates without ordering all the vertices, then the subgraph of the vertices that have not been ordered must contain a directed cycle.

```
1  def topological_sort(g):
2    """Return a list of verticies of directed acyclic graph g in topological order.
3
4    If graph g has a cycle, the result will be incomplete.
5    """
6    topo = [ ]                  # a list of vertices placed in topological order
7    ready = [ ]                 # list of vertices that have no remaining constraints
8    incount = { }               # keep track of in-degree for each vertex
9    for u in g.vertices( ):
10     incount[u] = g.degree(u, False)    # parameter requests incoming degree
11     if incount[u] == 0:                # if u has no incoming edges,
12       ready.append(u)                  # it is free of constraints
13   while len(ready) > 0:
14     u = ready.pop( )                   # u is free of constraints
15     topo.append(u)                     # add u to the topological order
16     for e in g.incident_edges(u):      # consider all outgoing neighbors of u
17       v = e.opposite(u)
18       incount[v] -= 1                  # v has one less constraint without u
19       if incount[v] == 0:
20         ready.append(v)
21   return topo
```

Code Fragment 14.11: Python implementation for the topological sorting algorithm. (We show an example execution of this algorithm in Figure 14.13.)

Performance of Topological Sorting

Proposition 14.22: *Let $\vec{G}$ be a directed graph with n vertices and m edges, using an adjacency list representation. The topological sorting algorithm runs in $O(n+m)$ time using $O(n)$ auxiliary space, and either computes a topological ordering of $\vec{G}$ or fails to include some vertices, which indicates that $\vec{G}$ has a directed cycle.*

Justification: The initial recording of the n in-degrees uses $O(n)$ time based on the degree method. Say that a vertex u is ***visited*** by the topological sorting algorithm when u is removed from the ready list. A vertex u can be visited only when incount(u) is 0, which implies that all its predecessors (vertices with outgoing edges into u) were previously visited. As a consequence, any vertex that is on a directed cycle will never be visited, and any other vertex will be visited exactly once. The algorithm traverses all the outgoing edges of each visited vertex once, so its running time is proportional to the number of outgoing edges of the visited vertices. In accordance with Proposition 14.9, the running time is $(n+m)$. Regarding the space usage, observe that containers topo, ready, and incount have at most one entry per vertex, and therefore use $O(n)$ space. ■

Chapter 4

Paths in graphs

4.1 Distances

Depth-first search readily identifies all the vertices of a graph that can be reached from a designated starting point. It also finds explicit paths to these vertices, summarized in its search tree (Figure 4.1). However, these paths might not be the most economical ones possible. In the figure, vertex C is reachable from S by traversing just one edge, while the DFS tree shows a path of length 3. This chapter is about algorithms for finding *shortest paths* in graphs.

Path lengths allow us to talk quantitatively about the extent to which different vertices of a graph are separated from each other:

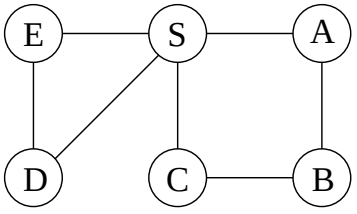
The *distance* between two nodes is the length of the shortest path between them.

To get a concrete feel for this notion, consider a physical realization of a graph that has a ball for each vertex and a piece of string for each edge. If you lift the ball for vertex s high enough, the other balls that get pulled up along with it are precisely the vertices reachable from s. And to find their distances from s, you need only measure how far below s they hang.

In Figure 4.2 for example, vertex B is at distance 2 from S, and there are two shortest paths to it. When S is held up, the strings along each of these paths become taut. On the other hand, edge (D, E) plays no role in any shortest path and therefore remains slack.

Figure 4.1 (a) A simple graph and (b) its depth-first search tree.

(a)



(b)

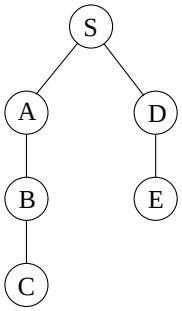


Figure 4.2 A physical model of a graph.

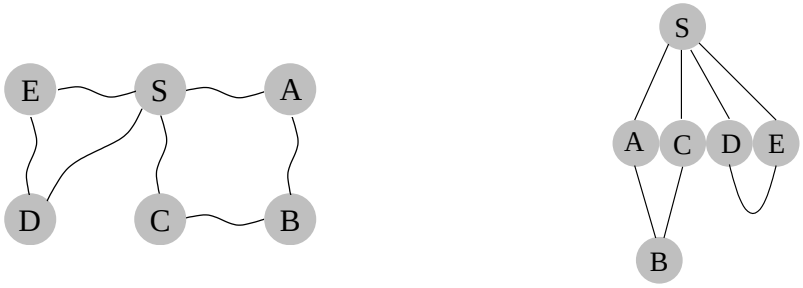


Figure 4.3 Breadth-first search.

```
procedure bfs(G, s)
Input:    Graph G = (V, E), directed or undirected; vertex s ∈ V
Output:   For all vertices u reachable from s, dist(u) is set
          to the distance from s to u.

for all u ∈ V:
   dist(u) = ∞

dist(s) = 0
Q = [s] (queue containing just s)
while Q is not empty:
   u = eject(Q)
   for all edges (u, v) ∈ E:
      if dist(v) = ∞:
         inject(Q, v)
         dist(v) = dist(u) + 1
```

4.2 Breadth-first search

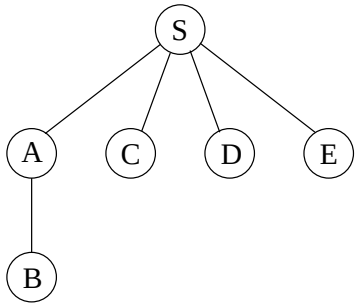
In Figure 4.2, the lifting of s partitions the graph into layers: s itself, the nodes at distance 1 from it, the nodes at distance 2 from it, and so on. A convenient way to compute distances from s to the other vertices is to proceed layer by layer. Once we have picked out the nodes at distance $0, 1, 2, \ldots, d$, the ones at $d + 1$ are easily determined: they are precisely the as-yet-unseen nodes that are adjacent to the layer at distance d. This suggests an iterative algorithm in which two layers are active at any given time: some layer d, which has been fully identified, and $d + 1$, which is being discovered by scanning the neighbors of layer d.

Breadth-first search (BFS) directly implements this simple reasoning (Figure 4.3). Initially the queue Q consists only of s, the one node at distance 0. And for each subsequent distance $d = 1, 2, 3, \ldots$, there is a point in time at which Q contains all the nodes at distance d and nothing else. As these nodes are processed (ejected off the front of the queue), their as-yet-unseen neighbors are injected into the end of the queue.

Let's try out this algorithm on our earlier example (Figure 4.1) to confirm that it does the

Figure 4.4 The result of breadth-first search on the graph of Figure 4.1.

Order of visitation	Queue contents after processing node
	$[S]$
S	$[A\ C\ D\ E]$
A	$[C\ D\ E\ B]$
C	$[D\ E\ B]$
D	$[E\ B]$
E	$[B]$
B	$[\,]$



right thing. If S is the starting point and the nodes are ordered alphabetically, they get visited in the sequence shown in Figure 4.4. The breadth-first search tree, on the right, contains the edges through which each node is initially discovered. Unlike the DFS tree we saw earlier, it has the property that all its paths from S are the shortest possible. It is therefore a *shortest-path tree*.

Correctness and efficiency

We have developed the basic intuition behind breadth-first search. In order to check that the algorithm works correctly, we need to make sure that it faithfully executes this intuition. What we expect, precisely, is that

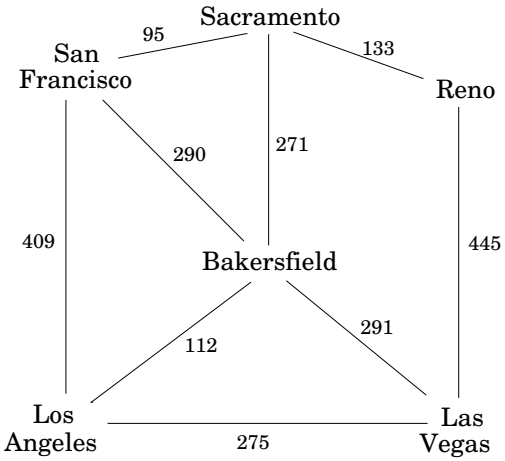
> For each $d = 0, 1, 2, \ldots$, there is a moment at which (1) all nodes at distance $\leq d$ from s have their distances correctly set; (2) all other nodes have their distances set to ∞; and (3) the queue contains exactly the nodes at distance d.

This has been phrased with an inductive argument in mind. We have already discussed both the base case and the inductive step. Can you fill in the details?

The overall running time of this algorithm is linear, $O(|V| + |E|)$, for exactly the same reasons as depth-first search. Each vertex is put on the queue exactly once, when it is first encountered, so there are $2\,|V|$ queue operations. The rest of the work is done in the algorithm's innermost loop. Over the course of execution, this loop looks at each edge once (in directed graphs) or twice (in undirected graphs), and therefore takes $O(|E|)$ time.

Now that we have both BFS and DFS before us: how do their exploration styles compare? Depth-first search makes deep incursions into a graph, retreating only when it runs out of new nodes to visit. This strategy gives it the wonderful, subtle, and extremely useful properties we saw in the Chapter 3. But it also means that DFS can end up taking a long and convoluted route to a vertex that is actually very close by, as in Figure 4.1. Breadth-first search makes sure to visit vertices in increasing order of their distance from the starting point. This is a broader, shallower search, rather like the propagation of a wave upon water. And it is achieved using almost exactly the same code as DFS—but with a queue in place of a stack.

Figure 4.5 Edge lengths often matter.



Also notice one stylistic difference from DFS: since we are only interested in distances from s, we do not restart the search in other connected components. Nodes not reachable from s are simply ignored.

4.3 Lengths on edges

Breadth-first search treats all edges as having the same length. This is rarely true in applications where shortest paths are to be found. For instance, suppose you are driving from San Francisco to Las Vegas, and want to find the quickest route. Figure 4.5 shows the major highways you might conceivably use. Picking the right combination of them is a shortest-path problem in which the length of each edge (each stretch of highway) is important. For the remainder of this chapter, we will deal with this more general scenario, annotating every edge $e \in E$ with a length l_e. If $e = (u, v)$, we will sometimes also write $l(u, v)$ or l_{uv}.

These l_e's do not have to correspond to physical lengths. They could denote time (driving time between cities) or money (cost of taking a bus), or any other quantity that we would like to conserve. In fact, there are cases in which we need to use negative lengths, but we will briefly overlook this particular complication.

4.4 Dijkstra's algorithm

4.4.1 An adaptation of breadth-first search

Breadth-first search finds shortest paths in any graph whose edges have unit length. Can we adapt it to a more general graph $G = (V, E)$ whose edge lengths l_e are *positive integers*?

A more convenient graph

Here is a simple trick for converting G into something BFS can handle: break G's long edges into unit-length pieces, by introducing "dummy" nodes. Figure 4.6 shows an example of this